
Reparameterization Flow Policy Optimization

Hai Zhong¹ Zhuoran Li¹ Xun Wang¹ Longbo Huang¹

Abstract

Reparameterization Policy Gradient (RPG) has emerged as a powerful paradigm for model-based reinforcement learning, enabling high sample efficiency by backpropagating gradients through differentiable dynamics. However, prior RPG approaches have been predominantly restricted to Gaussian policies, limiting their performance and failing to leverage recent advances in generative models. In this work, we identify that flow policies, which generate actions via differentiable ODE integration, naturally align with the RPG framework, a connection not established in prior work. However, naively exploiting this synergy proves ineffective, often suffering from training instability and a lack of exploration. We propose Reparameterization Flow Policy Optimization (RFO). RFO computes policy gradients by backpropagating jointly through the flow generation process and system dynamics, unlocking high sample efficiency without requiring intractable log-likelihood calculations. RFO includes two tailored regularization terms for stability and exploration. We also propose a variant of RFO with action chunking. Extensive experiments on diverse locomotion and manipulation tasks, involving both rigid and soft bodies with state or visual inputs, demonstrate the effectiveness of RFO. Notably, on a challenging locomotion task controlling a soft-body quadruped, RFO achieves almost $2\times$ the reward of the state-of-the-art baseline.

1. Introduction

Reparameterization Policy Gradient (RPG) parameterizes the policy as a differentiable mapping from a source noise distribution to actions (Kingma & Welling, 2014; Rezende

et al., 2014), allowing gradients to be backpropagated directly through the trajectory. Compared to likelihood-ratio estimators such as REINFORCE (Williams, 1992), RPG generally exhibits lower variance and thus higher sample efficiency (Gao et al., 2024; Xing et al., 2025). These advantages have fueled a surge of robotic applications built on differentiable simulators, spanning quadrupeds to quadrotors (Luo et al., 2025; Zhang et al., 2025b; Heeg et al., 2025; Schwarke et al., 2025; Pan et al., 2025).

Yet RPG has so far been studied mostly in the context of Gaussian policies, which restricts the family of action distributions the policy can express. In contrast, flow policies trained via imitation learning have recently demonstrated remarkable success in robotics (Black et al., 2025a; 2026; 2025b; Lipman et al., 2023), owing to their superior expressiveness, implementation simplicity, and fast inference. To overcome the limitations of demonstrations, there has been a surge of research interest in leveraging reinforcement learning (RL) to enable flow policies to learn directly from their own experience (Intelligence et al., 2025; McAllister et al., 2025; Zhang et al., 2025c;a; 2026; Lv et al., 2025).

In this work, we identify the key insight that flow policies inherently parameterize a differentiable mapping from source noise to actions, thus naturally aligning with the RPG framework. Leveraging this synergy, we view the flow generation process as an integral part of the rollout trajectory, enabling policy gradients to be computed by backpropagating directly through both the ODE integration and system dynamics, a novel connection not established in prior work. However, directly optimizing flow policies with RPG fails to yield strong performance, primarily due to optimization instability and insufficient exploration.

Hence, we propose **Reparameterization Flow Policy Optimization (RFO)**, an on-policy RL algorithm optimizing flow policies with RPG. Beyond the core RPG mechanism, we introduce two regularization terms crucial for stable training. The first leverages actions sampled during rollouts as targets for Conditional Flow Matching (CFM) to stabilize policy updates, while the second utilizes targets from the uniform distribution to explicitly encourage exploration. Additionally, we propose a variant of RFO incorporating an action chunking mechanism (Black et al., 2025a;b; Li et al., 2025b). We evaluate our method across a diverse suite

¹Institute for Interdisciplinary Information Sciences (IIIS), Tsinghua University. Correspondence to: Longbo Huang <longbo-huang@tsinghua.edu.cn>.

of locomotion and manipulation tasks, involving both rigid and soft bodies, using state or visual inputs. Notably, RFO achieves nearly $2\times$ the reward of the state-of-the-art (SOTA) baseline on a challenging soft-body quadruped locomotion task with a high-dimensional action space ($\mathbb{R}^{222}$).

RFO represents a meaningful contribution to both the flow-based RL and RPG communities. From the perspective of flow-based RL, RFO uniquely combines the strengths of distinct paradigms: it inherits the high sample efficiency of RPG while eliminating the need for intractable action log-likelihood computations. Simultaneously, as an on-policy approach, RFO seamlessly integrates with massively parallel differentiable physics simulators, such as Rewarped and DFlex (Xing et al., 2025; Xu et al., 2021; Georgiev et al., 2024; NVIDIA et al., 2025), to significantly accelerate training.

To summarize, our contributions are threefold: (i) We propose a novel paradigm for training flow policies using Reparameterization Policy Gradients, unlocking high sample efficiency without approximating intractable likelihoods. (ii) We introduce Reparameterization Flow Policy Optimization (RFO), an algorithm equipped with novel regularization mechanisms for stability and exploration, further extended with an action-chunking formulation. (iii) We conduct extensive experiments on differentiable simulators, demonstrating that RFO consistently achieves strong performance across challenging rigid and soft-body tasks compared to existing baselines.

2. Related Work

2.1. Online Reinforcement Learning Algorithms for Flow and Diffusion Policy

Online RL algorithms for flow and diffusion policies can be categorized as off-policy or on-policy. Off-policy algorithms rely on Q-functions to train generative policies (Psenka et al., 2024; Wang et al., 2025a; Celik et al., 2025; Lv et al., 2025; Ma et al., 2025; Chen et al., 2025; Dong et al., 2025; Yang et al., 2023; Zhang et al., 2025c; Ding et al., 2024; Wang et al., 2025b; Park et al., 2025), offering the advantage of bypassing the calculation of action log-likelihoods. One approach for policy optimization involves backpropagating Q-gradients to the noise prediction or vector field network (Wang et al., 2024; 2025a; Lv et al., 2025; Wang et al., 2025b; Zhang et al., 2025c; Celik et al., 2025). This is achieved by reparameterizing the flow or diffusion policy as a transformation from sampled noise to actions, which allows for the direct backpropagation of Q-function gradients through this differentiable transformation to the policy network. Another class of approaches exploits the intrinsic connection between the score function (or velocity field) and Q-functions (Psenka et al., 2024; Dong et al., 2025; Akhound-

Sadeh et al., 2024; Ding et al., 2024). For instance, (Psenka et al., 2024) proposes matching the score functions to Q-function gradients. Meanwhile, (Dong et al., 2025; Ma et al., 2025; Ding et al., 2024) utilize a Q-weighted regression loss to train the noise prediction network, whereas (Chen et al., 2025) trains the velocity network via a Q-weighted conditional flow matching loss.

On-policy RL algorithms for flow and diffusion policies (Ding et al., 2025; Ren et al., 2025; Zhang et al., 2025a; Yang et al., 2025; McAllister et al., 2025) must address the challenge of approximating otherwise intractable action log-likelihoods. One prevalent approach is to treat the sampled noise as an augmented state, which enables the calculation of action likelihoods conditioned on the noise. For instance, DPPO (Ren et al., 2025) constructs a two-level Markov Decision Process that propagates policy gradient signals to each denoising step. Similarly, NCDPO and Reinfo (Yang et al., 2025; Zhang et al., 2025a) treat the action generation as a deterministic process by conditioning on all sampled noise, and additionally train a noise injection network to compute action log-likelihoods. Alternatively, FPO (McAllister et al., 2025) approximates action log-likelihoods via the conditional flow matching loss. GenPO (Ding et al., 2025) introduces a specialized doubled dummy action generation mechanism, which enables the calculation of exact action log-likelihoods. In contrast, RFO entirely bypasses likelihood computation by leveraging reparameterization gradients through differentiable dynamics, which typically yields lower-variance gradient estimates than REINFORCE-like policy gradients (Mohamed et al., 2020).

2.2. Reparameterization Policy Gradient

Reparameterization Policy Gradient exploits the reparameterization trick (Kingma & Welling, 2014; Rezende et al., 2014; Mohamed et al., 2020) to render the action sampling process differentiable. By defining the action as a deterministic transformation of a noise variable drawn from a fixed source distribution, RPG enables the computation of low-variance policy gradients by backpropagating directly through the system dynamics. These dynamics are typically provided by either differentiable simulators (Xu et al., 2021; Hu et al., 2020; Xing et al., 2025) or learned world models (Hafner et al., 2020; Amos et al., 2021; Li et al., 2025a). Consequently, RPG generally yields gradient estimates with significantly lower variance compared to likelihood-ratio estimators like REINFORCE (Mohamed et al., 2020; Sutton et al., 1999). Prominent RPG algorithms, such as SHAC and AHAC (Xu et al., 2021; Georgiev et al., 2024), further stabilize training by truncating the backpropagation horizon and leveraging value function gradients to estimate long-term returns. While prior work has investigated the use of multimodal policies within RPG (Huang et al., 2023), the integration of modern expressive generative models,

specifically diffusion and flow matching, remains largely unexplored. RFO bridges this gap by introducing the first RPG-based framework tailored for flow policies.

3. Preliminaries

3.1. Reinforcement Learning Problem Formulation

We formulate the reinforcement learning (RL) problem as a Markov Decision Process (MDP) (Sutton & Barto, 2018), defined by the tuple $(\mathcal{S}, \mathcal{A}, p, r, \rho_0, \gamma)$. Here, $\mathcal{S}$ denotes the state space, and $\mathcal{A}$ denotes the action space. The transition function $p(s'|s, a)$ defines the probability density of transitioning to the next state $s' \in \mathcal{S}$ given the current state $s \in \mathcal{S}$ and action $a \in \mathcal{A}$. The term $r(s, a)$ represents the reward function, ρ_0 denotes the initial state distribution, and $\gamma \in [0, 1)$ is the discount factor. In this work, we consider a continuous and **bounded** action space $\mathcal{A}$. The objective of RL is to maximize the expected discounted cumulative reward:

$$J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta} \left[\sum_{t=0}^{\infty} \gamma^t r(s_t, a_t) \right], \quad (1)$$

where $\tau = (s_0, a_0, s_1, a_1, \dots)$ denotes a trajectory sampled from the distribution induced by the policy π_θ , with $s_0 \sim \rho_0$.

3.2. Flow Model and Flow Policy

Flow Models and Flow Matching. Flow models (Lipman et al., 2023; Chen et al., 2018; Albergo & Vanden-Eijnden, 2023; Liu et al., 2023) constitute a class of generative models that transform samples from a source distribution $x_0 \sim p_0$ (e.g., standard Gaussian) to a target distribution $x_1 \sim p_1$ via an invertible flow map $\phi : \mathbb{R}^d \times [0, 1] \rightarrow \mathbb{R}^d$. This flow map is induced by a time-dependent, neural-network-parameterized vector field $v_\theta(x, u)$ (Chen et al., 2018), defined by the ordinary differential equation (ODE):

$$\frac{d}{du} \phi(x_0, u) = v_\theta(\phi(x_0, u), u), \quad (2)$$

where $\phi(x_0, 0) = x_0$, u denotes the flow time, and $p_0(x_0)$ denotes the density of the source distribution. To learn the parameterized vector field v_θ , the Conditional Flow Matching (CFM) objective is proposed (Lipman et al., 2023). Specifically, CFM constructs a linear interpolation path conditioned on the source noise x_0 and the target sample x_1 , where the intermediate state is given by:

$$x_u = (1 - u)x_0 + ux_1, \quad u \in [0, 1]. \quad (3)$$

Consequently, the CFM objective is defined as:

$$\begin{aligned} \mathcal{L}_{\text{CFM}}(\theta) &= \mathbb{E}_{u \sim \mathcal{U}[0, 1], x_0 \sim p_0, x_1 \sim p_1} \left[\|v_\theta(x_u, u) - (x_1 - x_0)\|^2 \right], \end{aligned} \quad (4)$$

where $\mathcal{U}[0, 1]$ is the uniform distribution.

Flow Policy. Flow models serve as a powerful policy class for generating actions (Black et al., 2025a; Intelligence et al., 2025; Black et al., 2025b) from potentially complex, multimodal distributions. To define a state-conditioned policy, the vector field $v_\theta(x, u|s)$ is conditioned on the current state $s \in \mathcal{S}$. An action a is generated by first sampling x_0 from the source distribution p_0 , and subsequently solving the following ODE from $u = 0$ to $u = 1$:

$$\frac{dx}{du} = v_\theta(x, u|s), \quad \text{with } x(0) = x_0. \quad (5)$$

The core challenge in utilizing flow policies for RL lies in propagating reward signals to optimize the vector field v_θ .

3.3. Reparameterization Policy Gradient

RPG is grounded in the reparameterization trick (Kingma & Welling, 2014; Rezende et al., 2014). The key idea is to learn a deterministic transformation that maps a fixed source distribution into the desired action distribution. Concretely, with noise ϵ drawn from the source distribution, the action is generated by a learned function $a = f_\theta(\epsilon; s)$, typically a neural network, conditioned on the state s . To date, the literature has mostly instantiated this transformation as a Gaussian policy (Mohamed et al., 2020; Xu et al., 2021; Georgiev et al., 2024; Xing et al., 2025; Gao et al., 2024), mapping a standard Gaussian source to a state-dependent Gaussian distribution.

Following previous RPG work (Xu et al., 2021; Georgiev et al., 2024; Xing et al., 2025), we focus on deterministic environment dynamics $s_{t+1} = g(s_t, a_t)$ and assume environment dynamics and the reward functions are differentiable.

Leveraging the reparameterization transformation, RPG computes the policy gradient by backpropagating through the environment dynamics, utilizing the dynamics Jacobians $\frac{\partial s_{t+1}}{\partial a_t}$ and $\frac{\partial s_{t+1}}{\partial s_t}$. The objective function and its gradient are defined as:

$$J(\theta) = \mathbb{E}_{s_0, \epsilon_0, \epsilon_1, \dots} [R(\tau)], \quad (6)$$

$$\nabla_\theta J(\theta) = \mathbb{E}_{s_0, \epsilon_0, \epsilon_1, \dots} [\nabla_\theta R(\tau)], \quad (7)$$

where $R(\tau) = \sum_{t=0}^{\infty} \gamma^t r(s_t, a_t)$ denotes the cumulative discounted reward for the trajectory τ . Crucially, this expectation is taken with respect to the initial state distribution and the sequence of independent noise variables sampled from the source distribution at each time step.

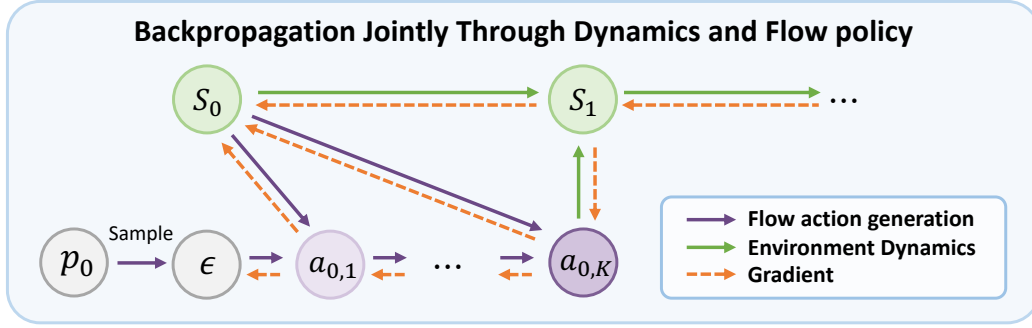


Figure 1. RFO optimizes the flow policy by jointly backpropagating through the dynamics and the flow policy.

4. Reparameterization Flow Policy Optimization

In this section, we detail the proposed algorithm, Reparameterization Flow Policy Optimization. RFO comprises three key components. First, we derive the optimization of flow policies with RPG. Second, we introduce two regularization terms that are critical for RFO’s performance. Finally, we present a variant of RFO that incorporates an action chunking mechanism.

4.1. Reparameterization Policy Gradient for Flow Policy

We now detail the first key component of RFO: reparameterization policy gradient for flow policies. We adopt the *discretize-then-optimize* paradigm (Onken & Ruthotto, 2020) for flow policy optimization, consistent with prior studies on reinforcement learning with flow policies (Zhang et al., 2025a; Lv et al., 2025; Zhang et al., 2025c; Ding et al., 2025). Specifically, given the neural-network-parameterized vector field v_θ , we perform numerical integration on the ODE using the Euler method (Eul, 2016). We employ the double-subscript notation $a_{t,k}$, where the subscript t denotes the MDP time step and k indexes the internal discrete integration steps. The generation process is formalized as follows:

$$\begin{aligned} a_{t,0} &= \epsilon_t, \quad \text{where } \epsilon_t \sim p_0, \\ a_{t,k+1} &= a_{t,k} + \Delta u \cdot v_\theta(a_{t,k}, u_k | s_t), \quad k = 0, \dots, K-1, \\ a_t &= a_{t,K}, \end{aligned} \quad (8)$$

where $\Delta u = 1/K$ is the step size and $u_k = k \cdot \Delta u$ denotes the k -th flow time. Consequently, the Euler integration process explicitly defines a differentiable transformation from the noise ϵ_t to the action a_t . We denote this transformation as $a_t = F_\theta(\epsilon_t; s_t)$. Based on this, we have the following key insight to optimize RFO.

Key Insight: Flow policies inherently parameterize a differentiable mapping from source noise to actions. This structure is naturally compatible with the Reparameterization Policy Gradient framework, enabling the computation

of policy gradients via end-to-end backpropagation jointly through the environment dynamics and the flow policy.

An illustration of this insight is shown in Figure 1. To formalize this insight, reparameterization policy gradient for the flow policy is expressed as:

$$\nabla_\theta J(\theta) = \mathbb{E}_{s_0, \epsilon_0, \epsilon_1, \dots} \left[\nabla_\theta \sum_{t=0}^{\infty} \gamma^t r(s_t, F_\theta(\epsilon_t; s_t)) \right], \quad (9)$$

where ϵ_t denotes the noise sampled from the source distribution at time t , and F_θ represents the flow transformation. The gradient is computed via backpropagation through time (BPTT) on the computational graph, which includes both the flow ODE integration and the environment dynamics typically provided by a differentiable simulator or a learned world model.

In practice, backpropagating through long horizons can lead to exploding gradients. Therefore, we follow the Short-Horizon Actor-Critic (SHAC) paradigm (Xu et al., 2021; Georgiev et al., 2024) by truncating trajectories into a batch of short segments to approximately compute gradient estimates for $J(\theta)$. That is, we backpropagate through a truncated horizon:

$$\nabla_\theta \left(\sum_{t=0}^{H-1} \gamma^t r(s_t, F_\theta(\epsilon_t; s_t)) + \gamma^H V_\omega(s_H) \right), \quad (10)$$

where s_0 represents the start state of the current segment, $V_\omega(s_H)$ is the terminal value estimate to account for future rewards, and ω denotes the parameters for the value function network. To rigorously estimate $\nabla_\theta J(\theta)$ at each iteration, one should re-sample initial states from ρ_0 and restart the rollout. In practice, however, we follow SHAC’s procedure and continue from the previous segment’s truncated state, which may differ from the initial state distribution, but this works well empirically. Following SHAC, the accumulated gradient is normalized by the product of the batch size and the rollout horizon H .

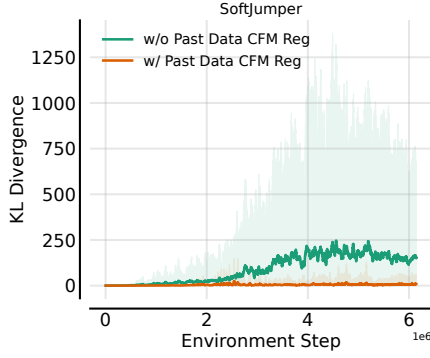


Figure 2. Comparison of RFO training with and without past data CFM regularization on the Soft Jumper task. We monitored the approximate KL divergence between consecutive policy updates.

4.2. Regularization Terms

RFO incorporates two regularization terms that are crucial for performance. The first term addresses the stability of policy updates. RPG optimizes the policy by backpropagating gradients through the ODE integration steps, which can potentially disrupt the flow trajectories towards previously generated actions, rendering them unlikely to be re-sampled under the updated policy.

To illustrate this phenomenon, we conducted experiments on the Soft Jumper task using the Rewarped simulator (Xing et al., 2025). We monitored the CFM loss of the current policy evaluated on actions sampled from the immediately preceding iteration. The CFM loss steadily increases and remains high, indicating that the updated vector field struggles to reconstruct past actions. Furthermore, we approximately tracked the KL divergence between consecutive policy updates (KL divergence approximation details are provided in Appendix E). As shown in Figure 2, the large KL divergence clearly signals training instability.

Algorithm 1 Reparameterization Flow Policy Optimization (RFO)

- 1: Initialize θ, ω and buffers $\mathcal{D}_{\text{recent}}, \mathcal{D}_{\text{rollout}}$.
- 2: **for** iteration $i = 1, 2, \dots$ **do**
- 3: Collect short-horizon trajectories with flow policy; update $\mathcal{D}_{\text{recent}}$ and $\mathcal{D}_{\text{rollout}}$.
- 4: Compute the RPG gradient for the flow policy via BPTT (Eq. (10)).
- 5: Compute regularization CFM loss gradients $\nabla \mathcal{L}_{\text{past}}$ (Eq. (11)) and $\nabla \mathcal{L}_{\text{uni}}$ (Eq. (12)).
- 6: Update flow policy with combined and weighted gradients (Eq. (14)).
- 7: Update critic ω for L epochs by minimizing Eq. (13).
- 8: **end for**

Hence, intuitively, we aim to regularize the vector field to retain paths from the source distribution to these recently

sampled actions. We propose the *Past Data CFM Regularization*. This term treats actions from the current and previous rollout iterations as target samples and ensures that the vector field points towards them. At training iteration i , let $\mathcal{D}_i$ and $\mathcal{D}_{i-1}$ denote the rollout batches collected during the current and previous iteration, respectively. The objective $\mathcal{L}_{\text{past}}$ is defined as:

$$\begin{aligned} \mathcal{L}_{\text{past}}(\theta) &= \sum_{j \in \{i-1, i\}} \mathbb{E}_{u \sim \mathcal{U}[0,1], \epsilon \sim p_0, (s,a) \sim \mathcal{D}_j} \left[\|v_\theta(\psi_u, u | s) - (a - \epsilon)\|^2 \right], \end{aligned} \quad (11)$$

where u is the flow time and $\psi_u = (1 - u)\epsilon + ua$ represents the linear interpolation between the source noise ϵ and the target action a . In implementation, the two iteration-specific CFM losses are computed separately and summed. We denote the corresponding pair of rollout batches by $\mathcal{D}_{\text{recent}} = \{\mathcal{D}_{i-1}, \mathcal{D}_i\}$. A discussion of this design choice is provided in Appendix F. The effectiveness of $\mathcal{L}_{\text{past}}$ is empirically validated in Section 5.3. While self-imitation strategies have appeared in diffusion RL contexts (Yang et al., 2025), to the best of our knowledge, we are the first to uncover the fundamental role of Past Data CFM within the paradigm of RPG-driven flow policy optimization. Our analysis establishes that this regularization serves as a critical stabilizer for RPG-based flow optimization.

We introduce a second regularization term to explicitly promote exploration. Unlike prior Maximum Entropy approaches (Dong et al., 2025; Wang et al., 2024; Celik et al., 2025), which rely on approximating the entropy of the generative policy, we adopt a more direct strategy. Given the bounded action space in our setting, we propose to guide the flow policy toward uniformly sampled target actions via the CFM objective. While QVPO (Ding et al., 2024) explores a similar uniform-target regularization for diffusion policies, it employs state-dependent weighting. In contrast, we adopt a simple approach that treats all states encountered during rollouts equally. The proposed uniform exploration CFM objective, denoted as $\mathcal{L}_{\text{uni}}$, is defined as:

$$\begin{aligned} \mathcal{L}_{\text{uni}}(\theta) &= \mathbb{E}_{u \sim \mathcal{U}[0,1], \epsilon \sim p_0, s \sim \mathcal{D}_{\text{rollout}}, a \sim p_{\text{uni}}} \left[\|v_\theta(\psi_u, u | s) - (a - \epsilon)\|^2 \right], \end{aligned} \quad (12)$$

where $\mathcal{D}_{\text{rollout}}$ denotes the set of states visited during the current iteration’s rollout, and p_{uni} represents the uniform distribution over the bounded action space. As before, ψ_u denotes the linear interpolation. The empirical effectiveness of $\mathcal{L}_{\text{uni}}$ is analyzed in Section 5.3.

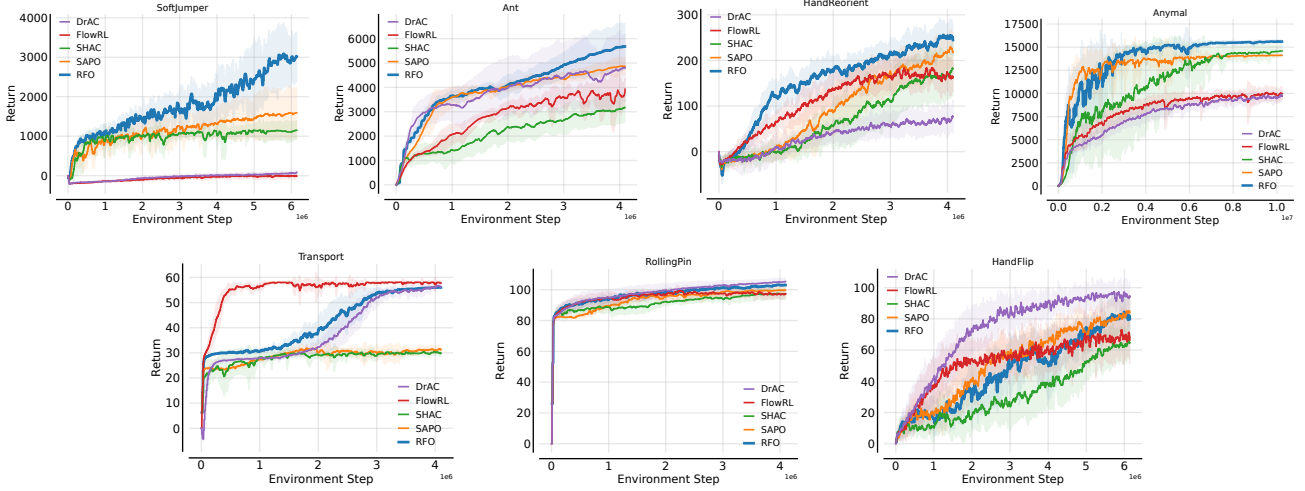


Figure 3. Training curves across seven tasks. Solid lines represent the mean return, while shaded regions indicate the standard deviation. Curves are smoothed using a 100-episode moving average.

4.3. Value Function Training

We optimize the critic by minimizing the standard mean squared error loss (Xing et al., 2025; Xu et al., 2021):

$$\mathcal{L}(\omega) = \mathbb{E} [\|V_\omega(s) - y(s)\|^2], \quad (13)$$

where $y(s)$ denotes the TD- λ target (Sutton & Barto, 2018). Following the dual-critic architecture in SAPO (Xing et al., 2025), we calculate the target value by averaging the predictions of two separate critic networks.

4.4. Overall Policy Objective and RFO Algorithm

Combining the RPG objective for the flow policy with the past data CFM regularization (Eq. (11)) and uniform exploration CFM regularization (Eq. (12)), RFO minimizes the total policy loss:

$$\mathcal{L}_{\text{policy}}(\theta) = -J(\theta) + c_{\text{past}}\mathcal{L}_{\text{past}}(\theta) + c_{\text{uni}}\mathcal{L}_{\text{uni}}(\theta), \quad (14)$$

where c_{past} and c_{uni} are hyperparameters weighting the stability and exploration regularization terms, respectively.

The overall RFO algorithm is described in Algorithm 1. We follow the SHAC paradigm, rolling out with flow policies to collect short-horizon trajectories. Then, we compute policy gradients and the gradients of the two CFM regularization terms. Combining and weighting these gradients, we update the policy. Afterwards, we update the value function by minimizing Eq. (13).

4.5. Action Chunking

Action chunking has proven beneficial for improving the temporal consistency of robot action generation (Black et al., 2025b), a critical factor in robotic applications. Therefore,

we provide an extension to RFO that incorporates action chunking. In this variant, the flow policy predicts an action chunk of size C based on the current observation, encompassing both current and future actions. Subsequently, the agent executes these actions for C steps before replanning based on the new observation.

5. Experiments

We conduct experiments to answer the following questions:

- (i) How does RFO’s performance compare to RPG-based approaches with Gaussian policies?
- (ii) How does RFO’s performance compare to other RL algorithms for flow or diffusion-based approaches?
- (iii) Are the two regularization terms critical to RFO’s performance?
- (iv) How does RFO work with the action chunking mechanism?

5.1. Experimental Setup

We evaluate our method on a comprehensive set of locomotion and manipulation tasks, utilizing either state or visual inputs within the differentiable simulators Rewarped (Xing et al., 2025) and DFlex (Xu et al., 2021; Georgiev et al., 2024).

Tasks. Our experiments span diverse rigid and soft-body dynamics, categorized into:

- Locomotion tasks: Ant, ANYmal (Hutter et al., 2017) (rigid quadruped), and Soft Jumper (visual soft body).
- Manipulation tasks: Hand Reorient (cube rotation), Rolling Pin (dough flattening), Hand Flip (object flipping), and Transport (liquid transport).

Visual inputs are used for Soft Jumper and the last three

Table 1. Final performance evaluation after training. Results are reported as mean $\pm$ standard deviation across all runs. Each run is evaluated over 128 episodes. Normalized score = $\frac{\text{Mean Reward}}{\text{SHAC Mean Reward}}$.

(a) Raw Final Evaluation Rewards

Task	DrAC	FlowRL	SHAC	SAPO	RFO (Ours)
Soft Jumper	83.56 $\pm$ 67.60	-75.71 $\pm$ 139.54	1148.87 $\pm$ 233.84	1597.99 $\pm$ 653.87	3023.96 $\pm$ 603.61
Ant	4846.41 $\pm$ 1412.76	4083.45 $\pm$ 1082.79	3143.08 $\pm$ 569.50	4865.36 $\pm$ 363.77	5677.88 $\pm$ 964.49
Hand Reorient	73.74 $\pm$ 34.22	135.57 $\pm$ 39.34	174.63 $\pm$ 57.54	213.44 $\pm$ 33.95	258.84 $\pm$ 21.80
ANYmal	9554.46 $\pm$ 928.33	9717.58 $\pm$ 1619.35	14568.97 $\pm$ 652.72	14095.90 $\pm$ 82.02	15622.76 $\pm$ 200.87
Transport	57.07 $\pm$ 0.85	58.04 $\pm$ 0.19	30.00 $\pm$ 1.43	31.35 $\pm$ 1.24	56.00 $\pm$ 0.28
Rolling Pin	104.71 $\pm$ 2.00	92.49 $\pm$ 2.88	97.29 $\pm$ 3.44	99.79 $\pm$ 2.42	103.11 $\pm$ 1.26
Hand Flip	89.82 $\pm$ 4.59	54.80 $\pm$ 18.55	65.57 $\pm$ 11.75	83.00 $\pm$ 7.49	82.65 $\pm$ 11.65

(b) SHAC-normalized Final Evaluation Rewards

Task	DrAC	FlowRL	SHAC	SAPO	RFO (Ours)
Soft Jumper	0.07 $\pm$ 0.06	-0.07 $\pm$ 0.12	1.00 $\pm$ 0.20	1.39 $\pm$ 0.57	2.63 $\pm$ 0.53
Ant	1.54 $\pm$ 0.45	1.30 $\pm$ 0.34	1.00 $\pm$ 0.18	1.55 $\pm$ 0.12	1.81 $\pm$ 0.31
Hand Reorient	0.42 $\pm$ 0.20	0.78 $\pm$ 0.23	1.00 $\pm$ 0.33	1.22 $\pm$ 0.19	1.48 $\pm$ 0.12
ANYmal	0.66 $\pm$ 0.06	0.67 $\pm$ 0.11	1.00 $\pm$ 0.04	0.97 $\pm$ 0.01	1.07 $\pm$ 0.01
Transport	1.90 $\pm$ 0.03	1.93 $\pm$ 0.01	1.00 $\pm$ 0.05	1.05 $\pm$ 0.04	1.87 $\pm$ 0.01
Rolling Pin	1.08 $\pm$ 0.02	0.95 $\pm$ 0.03	1.00 $\pm$ 0.04	1.03 $\pm$ 0.02	1.06 $\pm$ 0.01
Hand Flip	1.37 $\pm$ 0.07	0.84 $\pm$ 0.28	1.00 $\pm$ 0.18	1.27 $\pm$ 0.11	1.26 $\pm$ 0.18
Average	1.01	0.91	1.00	1.21	1.60

manipulation tasks.

Baselines. We compare RFO against strong baselines from two categories:

- RPG Baselines: SAPO (Xing et al., 2025) (SOTA RPG-based method with maximum entropy) and SHAC (Xu et al., 2021) (short-horizon RPG).
- Flow/Diffusion RL Baselines: DrAC (Wang et al., 2025b) (a SOTA diffusion policy) and FlowRL (Lv et al., 2025) (a SOTA flow RL method).

All tasks are evaluated using at least 10 random seeds for each algorithm. Further experiment details are provided in Appendix B.

5.2. Main Experiment Results

We show the training curves in Figure 3 and final performance evaluation results in Table 1.

In the Soft Jumper, Ant, ANYmal, and Hand Reorient tasks, RFO outperforms all baselines by substantial margins. Most notably, in the Soft Jumper task, RFO achieves a final reward nearly $2\times$ that of the best baseline, underscoring its capability in high-dimensional control settings (action space $\mathbb{R}^{222}$). This superiority extends to rigid-body dynamics, where RFO consistently dominates strong baselines, securing performance gains of approximately 21% on Hand Reorient and 17% on Ant, while boosting the reward by

over 1000 points on the ANYmal task. Furthermore, RFO demonstrates remarkable robustness on the Rolling Pin and Transport tasks, achieving performance comparable to the state-of-the-art baselines. As shown in the aggregate performance analysis (Table 1 (b)), which normalizes scores against the SHAC baseline, RFO delivers superior overall performance across the diverse task suite.

From the RPG perspective, RFO consistently improves upon SHAC and SAPO, validating the efficacy of integrating flow policies into the RPG framework. Compared to Flow and Diffusion baselines, RFO demonstrates a clear advantage by leveraging analytical gradients from RPG. This becomes particularly evident in the Soft Jumper, ANYmal, and Hand Reorient tasks, where DrAC and FlowRL struggle, while RFO demonstrates superior performance. Additionally, DrAC requires 20 denoising steps for action sampling, which not only slows down inference but also significantly increases training time compared to RFO, since it needs to backpropagate through all 20 action generation steps. Furthermore, despite being an on-policy algorithm, RFO achieves sample efficiency comparable to off-policy flow-based methods thanks to RPG, while avoiding the need to approximate intractable action log-likelihoods.

5.3. Ablation Study

In this section, we conduct ablation experiments to answer the following questions: (i) Are the two proposed regularization terms, past data CFM regularization and uniform expo-

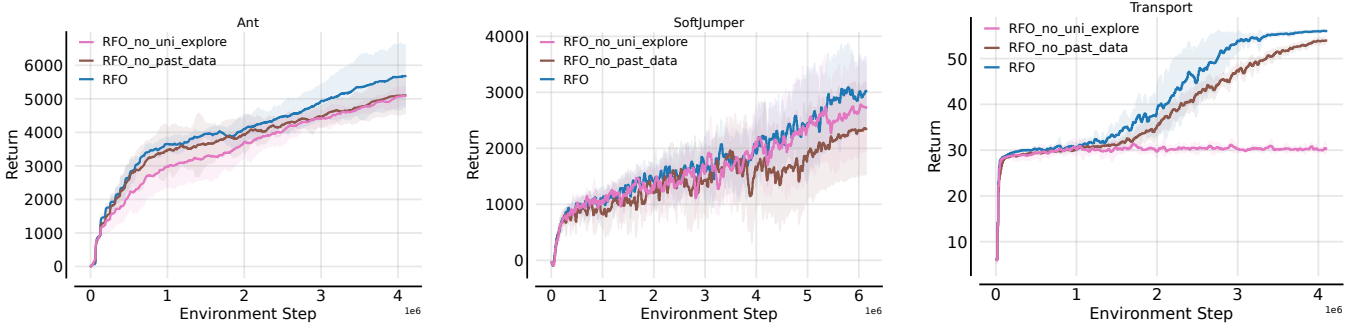


Figure 4. Ablation study for the effectiveness of past data CFM regularization and uniform exploration CFM regularization. The results show that the two proposed regularization terms are critical for RFO’s performance.

ration CFM regularization, critical for RFO’s performance?
 (ii) How does RFO perform under a range of hyperparameters, including the weights for the two CFM objectives and the number of flow integration steps?

To answer the first question, we conduct experiments on the Ant, Soft Jumper, and Transport tasks. We compare the full RFO algorithm against two variants: RFO without past data CFM regularization and RFO without uniform exploration CFM regularization. The training curves are presented in Figure 4.

Without past data CFM regularization, RFO’s performance significantly degrades across all three tasks, suggesting that this term is crucial for both training stability and final performance. Similarly, removing the uniform exploration regularization leads to performance drops across all tasks. Most notably, on the Transport task, omitting uniform exploration regularization causes RFO’s performance to degrade to the levels of SHAC and SAPO, clearly highlighting the effectiveness of the uniform random exploration strategy.

To answer the second question, we conduct experiments on the Soft Jumper task. We evaluate RFO with different combinations of c_{past} and c_{uni} , which control the weights of the two regularization terms. We also evaluate RFO with varying numbers of flow Euler integration steps. The training curves are provided in Appendix C, demonstrating that RFO exhibits robustness across a wide range of hyperparameters.

5.4. Action Chunking Results

Figure 6 in Appendix D illustrates the training curves for RFO with action chunking across the evaluated tasks. The flow policy must predict action sequences for future time steps based solely on the current observation, without the guidance of expert data used in imitation learning. Such optimization is harder, as the policy must generate valid future actions without access to the corresponding intermediate states. Consequently, this setup suggests that integrating offline-to-online procedures could be a promising direction

to further enhance RPG-based flow policies.

5.5. Memory Usage and Training Wall-Clock Time

We report the computational cost of RFO on the Soft Jumper task, which uses pixel inputs and is therefore representative of the more demanding settings in our benchmark. All experiments are conducted on a machine with 4 NVIDIA A40 GPUs, with one GPU dedicated to each seed. We run 4 seeds for RFO as well as the baselines SAPO (Xing et al., 2025), FlowRL (Lv et al., 2025), and DrAC (Wang et al., 2025b) for reference. Table 2 summarizes inference time per action, training throughput (environment steps per second), wall-clock training time, and GPU memory usage, reported as mean [min, max] across the four seeds.

Training Time. RFO trains for 2.68 hours on this task, comparable to the Gaussian-based RPG baseline SAPO (2.36 hours), with only 0.32 hour of additional overhead. This indicates that replacing the Gaussian policy with a flow policy under the RPG framework introduces only a modest training-time cost.

Memory. RFO uses 7.10 GB of GPU memory, 14.8% of an A40’s 48 GB capacity. We consider this a reasonable cost given RFO’s algorithmic performance.

Inference Latency. RFO requires 2.83 ms per action with 4-step flow integration, suitable for real-time control applications.

6. Conclusion

In this work, we introduce Reparameterization Flow Policy Optimization, a novel framework that bridges the gap between flow-based generative policies and the high sample efficiency of Reparameterization Policy Gradients. By integrating tailored regularization terms for training stability and exploration, RFO achieves strong performance. Empirical evaluations across diverse rigid and soft-body control tasks demonstrate that RFO achieves SOTA performance.

Table 2. Computational efficiency on the Soft Jumper task. Values are reported as mean [min, max] across 4 seeds.

Method	Inference Time (ms)	Training Throughput (steps/sec)	Training Time (h)	GPU Memory (GB)
SAPO	0.97 [0.94, 1.21]	727 [710, 743]	2.36 [2.31, 2.40]	4.23
FlowRL	1.22 [1.19, 1.40]	56 [56, 57]	30.41 [30.26, 30.60]	1.50
RFO (Ours)	2.83 [2.74, 2.96]	635 [629, 642]	2.68 [2.66, 2.70]	7.10
DrAC	8.73 [8.39, 9.50]	107 [105, 108]	16.04 [15.89, 16.20]	2.33

We further explored an action-chunking RFO variant. Future directions include extending RFO to offline-to-online reinforcement learning settings.

Impact Statement

This paper presents work whose goal is to advance the field of Machine Learning. There are many potential societal consequences of our work, none of which we feel must be specifically highlighted here.

Acknowledgements

This work was supported by the National Natural Science Foundation of China Grant 52494974.

References

- Numerical Differential Equation Methods*, chapter 2, pp. 55–142. John Wiley & Sons, Ltd, 2016. ISBN 9781119121534. doi: <https://doi.org/10.1002/9781119121534.ch2>. URL <https://onlinelibrary.wiley.com/doi/abs/10.1002/9781119121534.ch2>.
- Akhound-Sadegh, T., Rector-Brooks, J., Joey Bose, A., Mittal, S., Lemos, P., Liu, C.-H., Sendera, M., Ravanbakhsh, S., Gidel, G., Bengio, Y., Malkin, N., and Tong, A. Iterated denoising energy matching for sampling from boltzmann densities. In *Proceedings of the 41st International Conference on Machine Learning, ICML’24*. JMLR.org, 2024.
- Albergo, M. S. and Vanden-Eijnden, E. Building normalizing flows with stochastic interpolants. In *The Eleventh International Conference on Learning Representations*, 2023. URL <https://openreview.net/forum?id=li7qeBbCR1t>.
- Amos, B., Stanton, S., Yarats, D., and Wilson, A. G. On the model-based stochastic value gradient for continuous reinforcement learning. In Jadbabaie, A., Lygeros, J., Pappas, G. J., and Parrilo, P., Recht, B., Tomlin, C. J., and Zeilinger, M. N. (eds.), *Proceedings of the 3rd Conference on Learning for Dynamics and Control*, volume 144 of *Proceedings of Machine Learning Research*, pp. 6–20. PMLR, 07 – 08 June 2021. URL <https://proceedings.mlr.press/v144/amos21a.html>.
- Black, K., Brown, N., Darpinian, J., Dhabalia, K., Driess, D., Esmail, A., Equi, M. R., Finn, C., Fusai, N., Galliker, M. Y., Ghosh, D., Groom, L., Hausman, K., Ichter, B., Jakubczak, S., Jones, T., Ke, L., LeBlanc, D., Levine, S., Li-Bell, A., Mothukuri, M., Nair, S., Pertsch, K., Ren, A. Z., Shi, L. X., Smith, L., Springenberg, J. T., Stachowicz, K., Tanner, J., Vuong, Q., Walke, H., Walling, A., Wang, H., Yu, L., and Zhilinsky, U. $\pi_{0.5}$: a vision-language-action model with open-world generalization. In Lim, J., Song, S., and Park, H.-W. (eds.), *Proceedings of The 9th Conference on Robot Learning*, volume 305 of *Proceedings of Machine Learning Research*, pp. 17–40. PMLR, 27–30 Sep 2025a. URL <https://proceedings.mlr.press/v305/black25a.html>.
- Black, K., Galliker, M. Y., and Levine, S. Real-time execution of action chunking flow policies. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2025b. URL <https://openreview.net/forum?id=UkR2zO5uww>.
- Black, K., Brown, N., Driess, D., Esmail, A., Equi, M., Finn, C., Fusai, N., Groom, L., Hausman, K., Ichter, B., Jakubczak, S., Jones, T., Ke, L., Levine, S., Li-Bell, A., Mothukuri, M., Nair, S., Pertsch, K., Shi, L. X., Tanner, J., Vuong, Q., Walling, A., Wang, H., and Zhilinsky, U. π_0 : A vision-language-action flow model for general robot control, 2026. URL <https://arxiv.org/abs/2410.24164>.
- Celik, O., Li, Z., Blessing, D., Li, G., Palenicek, D., Peters, J., Chalvatzaki, G., and Neumann, G. DIME: Diffusion-based maximum entropy reinforcement learning. In *Forty-second International Conference on Machine Learning*, 2025. URL <https://openreview.net/forum?id=Aw6dBR7Vxj>.
- Chen, R. T. Q., Rubanova, Y., Bettencourt, J., and Duvenaud, D. K. Neural ordinary differential equations. In Bengio, S., Wallach, H., Larochelle, H., Grauman, K., Cesa-Bianchi, N., and Garnett, R. (eds.), *Advances in Neural Information Processing Systems*, volume 31. Curran Associates, Inc., 2018. URL <https://proceedings.neurips>.

- cc/paper_files/paper/2018/file/69386f6bb1dfed68692a24c8686939b9-Paper.pdf.
- Chen, T., Ma, H., Li, N., Wang, K., and Dai, B. One-step flow policy mirror descent, 2025. URL <https://arxiv.org/abs/2507.23675>.
- Ding, S., Hu, K., Zhang, Z., Ren, K., Zhang, W., Yu, J., Wang, J., and Shi, Y. Diffusion-based reinforcement learning via q-weighted variational policy optimization. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*, 2024. URL <https://openreview.net/forum?id=UWUUVKtKeu>.
- Ding, S., Hu, K., Zhong, S., Luo, H., Zhang, W., Wang, J., Wang, J., and Shi, Y. GenPO: Generative diffusion models meet on-policy reinforcement learning. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2025. URL <https://openreview.net/forum?id=BmRNz1TpCc>.
- Dong, X., Cheng, J., and Zhang, X. S. Maximum entropy reinforcement learning with diffusion policy. In *Forty-second International Conference on Machine Learning*, 2025. URL <https://openreview.net/forum?id=CpjKXe9rY7>.
- Gao, F., Shi, L., Zhang, S., Wang, Z., and Wu, Y. Adaptive-gradient policy optimization: Enhancing policy learning in non-smooth differentiable simulations. In *Forty-first International Conference on Machine Learning*, 2024. URL <https://openreview.net/forum?id=S9DV6ZP4eE>.
- Georgiev, I., Srinivasan, K., Xu, J., Heiden, E., and Garg, A. Adaptive horizon actor-critic for policy learning in contact-rich differentiable simulation. In *Proceedings of the 41st International Conference on Machine Learning*, ICML’24. JMLR.org, 2024.
- Hafner, D., Lillicrap, T., Ba, J., and Norouzi, M. Dream to control: Learning behaviors by latent imagination. In *International Conference on Learning Representations*, 2020. URL <https://openreview.net/forum?id=S1lOTC4tDS>.
- Heeg, J., Song, Y., and Scaramuzza, D. Learning quadrotor control from visual features using differentiable simulation. In *2025 IEEE International Conference on Robotics and Automation (ICRA)*, pp. 4033–4039, 2025. doi: 10.1109/ICRA55743.2025.11128641.
- Hu, Y., Anderson, L., Li, T.-M., Sun, Q., Carr, N., Ragan-Kelley, J., and Durand, F. DiffTaichi: Differentiable programming for physical simulation. *ICLR*, 2020.
- Huang, Z., Liang, L., Ling, Z., Li, X., Gan, C., and Su, H. Reparameterized policy learning for multimodal trajectory optimization. In *Proceedings of the 40th International Conference on Machine Learning*, ICML’23. JMLR.org, 2023.
- Hutter, M., Gehring, C., Lauber, A., Gunther, F., Bellicoso, C. D., Tsounis, V., Fankhauser, P., Diethelm, R., Bachmann, S., Bloesch, M., Kolvenbach, H., Bjelonic, M., Isler, L., and and, K. M. Anymal - toward legged robots for harsh environments. *Advanced Robotics*, 31(17):918–931, 2017. doi: 10.1080/01691864.2017.1378591. URL <https://doi.org/10.1080/01691864.2017.1378591>.
- Intelligence, P., Amin, A., Aniceto, R., Balakrishna, A., Black, K., Conley, K., Connors, G., Darpinian, J., Dhabalia, K., DiCarlo, J., Driess, D., Equi, M., Esmail, A., Fang, Y., Finn, C., Glossop, C., Godden, T., Goryachev, I., Groom, L., Hancock, H., Hausman, K., Hussein, G., Ichter, B., Jakubczak, S., Jen, R., Jones, T., Katz, B., Ke, L., Kuchi, C., Lamb, M., LeBlanc, D., Levine, S., Li-Bell, A., Lu, Y., Mano, V., Mothukuri, M., Nair, S., Pertsch, K., Ren, A. Z., Sharma, C., Shi, L. X., Smith, L., Springenberg, J. T., Stachowicz, K., Stoeckle, W., Swerdlow, A., Tanner, J., Torne, M., Vuong, Q., Walling, A., Wang, H., Williams, B., Yoo, S., Yu, L., Zhilinsky, U., and Zhou, Z. $\pi_{0.6}^*$: a vla that learns from experience, 2025. URL <https://arxiv.org/abs/2511.14759>.
- Kingma, D. P. and Welling, M. Auto-encoding variational Bayes. In *International Conference on Learning Representations (ICLR)*, 2014.
- Li, C., Krause, A., and Hutter, M. Robotic world model: A neural network simulator for robust policy optimization in robotics. *arXiv preprint arXiv:2501.10100*, 2025a.
- Li, Q., Zhou, Z., and Levine, S. Reinforcement learning with action chunking. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2025b. URL <https://openreview.net/forum?id=XUks1Y96NR>.
- Lipman, Y., Chen, R. T. Q., Ben-Hamu, H., Nickel, M., and Le, M. Flow matching for generative modeling. In *The Eleventh International Conference on Learning Representations*, 2023. URL <https://openreview.net/forum?id=PqvMRDCJT9t>.
- Liu, X., Gong, C., and qiang liu. Flow straight and fast: Learning to generate and transfer data with rectified flow. In *The Eleventh International Conference on Learning Representations*, 2023. URL <https://openreview.net/forum?id=XVjTTlnw5z>.

- Luo, J. Y., Song, Y., Klemm, V., Shi, F., Scaramuzza, D., and Hutter, M. Residual policy learning for perceptive quadruped control using differentiable simulation. In *2025 IEEE International Conference on Robotics and Automation (ICRA)*, pp. 1–8, 2025. doi: 10.1109/ICRA55743.2025.11127448.
- Lv, L., Li, Y., Luo, Y., Sun, F., Kong, T., Xu, J., and Ma, X. Flow-based policy for online reinforcement learning. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2025. URL <https://openreview.net/forum?id=CANUXhPoyN>.
- Ma, H., Chen, T., Wang, K., Li, N., and Dai, B. Efficient online reinforcement learning for diffusion policy. In *Forty-second International Conference on Machine Learning*, 2025. URL <https://openreview.net/forum?id=6Anv3KB9lz>.
- McAllister, D., Ge, S., Yi, B., Kim, C. M., Weber, E., Choi, H., Feng, H., and Kanazawa, A. Flow matching policy gradients, 2025. URL <https://arxiv.org/abs/2507.21053>.
- Mohamed, S., Rosca, M., Figurnov, M., and Mnih, A. Monte carlo gradient estimation in machine learning. *J. Mach. Learn. Res.*, 21(1), January 2020. ISSN 1532-4435.
- NVIDIA, :, Mittal, M., Roth, P., Tigue, J., Richard, A., Zhang, O., Du, P., Serrano-Muñoz, A., Yao, X., Zurbrugg, R., Rudin, N., Wawrzyniak, L., Rakhsha, M., Denzler, A., Heiden, E., Borovicka, A., Ahmed, O., Akinola, I., Anwar, A., Carlson, M. T., Feng, J. Y., Garg, A., Gasoto, R., Gulich, L., Guo, Y., Gussert, M., Hansen, A., Kulkarni, M., Li, C., Liu, W., Makoviychuk, V., Malczyk, G., Mazhar, H., Moghani, M., Murali, A., Noseworthy, M., Poddubny, A., Ratliff, N., Rehberg, W., Schwarke, C., Singh, R., Smith, J. L., Tang, B., Thaker, R., Trepte, M., Wyk, K. V., Yu, F., Millane, A., Ramasamy, V., Steiner, R., Subramanian, S., Volk, C., Chen, C., Jawale, N., Kuruttukulam, A. V., Lin, M. A., Mandlekar, A., Patzwaldt, K., Welsh, J., Zhao, H., Anes, F., Lafleche, J.-F., Moënnelocoz, N., Park, S., Stepinski, R., Gelder, D. V., Amevor, C., Carius, J., Chang, J., Chen, A. H., de Heras Ciechomski, P., Daviet, G., Mohajerani, M., von Mural, J., Reutsky, V., Sauter, M., Schirm, S., Shi, E. L., Terdiman, P., Vilella, K., Widmer, T., Yeoman, G., Chen, T., Grizan, S., Li, C., Li, L., Smith, C., Wiltz, R., Alexis, K., Chang, Y., Chu, D., Fan, L. J., Farshidian, F., Handa, A., Huang, S., Hutter, M., Narang, Y., Pouya, S., Sheng, S., Zhu, Y., Macklin, M., Moravanszky, A., Reist, P., Guo, Y., Hoeller, D., and State, G. Isaac lab: A gpu-accelerated simulation framework for multi-modal robot learning, 2025. URL <https://arxiv.org/abs/2511.04831>.
- Onken, D. and Ruthotto, L. Discretize-optimize vs. optimize-discretize for time-series regression and continuous normalizing flows, 2020. URL <https://arxiv.org/abs/2005.13420>.
- Pan, J., Xing, J., Reiter, R., Zhai, Y., Aljalbout, E., and Scaramuzza, D. Learning on the fly: Rapid policy adaptation via differentiable simulation. *IEEE Robotics and Automation Letters*, 2025.
- Park, S., Li, Q., and Levine, S. Flow q-learning. In *International Conference on Machine Learning (ICML)*, 2025.
- Psenka, M., Escontrela, A., Abbeel, P., and Ma, Y. Learning a diffusion model policy from rewards via q-score matching. In *Proceedings of the 41st International Conference on Machine Learning, ICML’24*. JMLR.org, 2024.
- Ren, A. Z., Lidard, J., Ankile, L. L., Simeonov, A., Agrawal, P., Majumdar, A., Burchfiel, B., Dai, H., and Simchowitz, M. Diffusion policy policy optimization. In *The Thirteenth International Conference on Learning Representations*, 2025. URL <https://openreview.net/forum?id=mEpqHvbD2h>.
- Rezende, D. J., Mohamed, S., and Wierstra, D. Stochastic backpropagation and approximate inference in deep generative models. In *Proceedings of The 30th International Conference on Machine Learning (ICML)*, 2014.
- Schulman, J. Approximating KL divergence, 2020. URL <http://joschu.net/blog/kl-approx.html>. <http://joschu.net/blog/kl-approx.html>.
- Schwarke, C., Klemm, V., Bagajo, J., Sleiman, J. P., Georgiev, I., Torres, J. T., and Hutter, M. Learning deployable locomotion control via differentiable simulation. In *9th Annual Conference on Robot Learning*, 2025. URL <https://openreview.net/forum?id=KHWIHwnYbn>.
- Sutton, R. S. and Barto, A. G. *Reinforcement Learning: An Introduction*. A Bradford Book, Cambridge, MA, USA, 2018. ISBN 0262039249.
- Sutton, R. S., McAllester, D., Singh, S., and Mansour, Y. Policy gradient methods for reinforcement learning with function approximation. In Solla, S., Leen, T., and Müller, K. (eds.), *Advances in Neural Information Processing Systems*, volume 12. MIT Press, 1999. URL https://proceedings.neurips.cc/paper_files/paper/1999/file/464d828b85b0bed98e80ade0a5c43b0f-Paper.pdf.

- Wang, Y., Wang, L., Jiang, Y., Zou, W., Liu, T., Song, X., Wang, W., Xiao, L., WU, J., Duan, J., and Li, S. E. Diffusion actor-critic with entropy regulator. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*, 2024. URL <https://openreview.net/forum?id=l0c1j4QvTq>.
- Wang, Y., Wang, L., Tan, M., Zou, W., Song, X., Wang, W., Liu, T., Zhan, G., Zhu, T., Liu, S., He, Z., Zhang, F., Duan, J., and Li, S. E. Enhanced dacer algorithm with high diffusion efficiency, 2025a. URL <https://arxiv.org/abs/2505.23426>.
- Wang, Z., Liu, J., and Pan, L. Learning intractable multimodal policies with reparameterization and diversity regularization. In *Annual Conference on Neural Information Processing Systems*, 2025b.
- Williams, R. J. Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine learning*, 8:229–256, 1992.
- Xing, E., Luk, V., and Oh, J. Stabilizing reinforcement learning in differentiable multiphysics simulation. In *The Thirteenth International Conference on Learning Representations*, 2025. URL <https://openreview.net/forum?id=DRiLWb8bJg>.
- Xu, J., Makovychuk, V., Narang, Y., Ramos, F., Matusik, W., Garg, A., and Macklin, M. Accelerated policy learning with parallel differentiable simulation. In *International Conference on Learning Representations*, 2021.
- Yang, L., Huang, Z., Lei, F., Zhong, Y., Yang, Y., Fang, C., Wen, S., Zhou, B., and Lin, Z. Policy representation via diffusion probability model for reinforcement learning, 2023. URL <https://arxiv.org/abs/2305.13122>.
- Yang, N., Gao, J., Gao, F., Wu, Y., and Yu, C. Fine-tuning diffusion policies with backpropagation through diffusion timesteps, 2025. URL <https://arxiv.org/abs/2505.10482>.
- Zhang, C., Wan, Z., Chen, F., Yang, F., Feng, L., Zhou, Y., Yu, X., You, Y., Tsang, I., and An, B. Evolving diffusion and flow matching policies for online reinforcement learning, 2026. URL <https://arxiv.org/abs/2512.02581>.
- Zhang, T., Yu, C., Su, S., and Wang, Y. Reinfrow: Fine-tuning flow matching policy with online reinforcement learning. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2025a. URL <https://openreview.net/forum?id=ACagRwCCqu>.
- Zhang, Y., Hu, Y., Song, Y., Zou, D., and Lin, W. Learning vision-based agile flight via differentiable physics. *Nature Machine Intelligence*, pp. 1–13, 2025b.
- Zhang, Y., Yu, S., Zhang, T., Guang, M., Hui, H., Long, K., Wang, Y., Yu, C., and Ding, W. Sac flow: Sample-efficient reinforcement learning of flow-based policies via velocity-reparameterized sequential modeling, 2025c. URL <https://arxiv.org/abs/2509.25756>.

A. Task Details

We conduct experiments with two differentiable physics simulators: DFlex (Xu et al., 2021; Georgiev et al., 2024) from NVIDIA and Rewarped (Xing et al., 2025). More specifically, we use AHAC’s version of the DFlex simulator and Rewarped version 1.3.0 from <https://github.com/rewarped/rewarped>.

Here, we provide details of the tasks used in the main experiments.

A.1. Ant

Ant is a classical locomotion task using the Rewarped simulator (Xing et al., 2025), where the goal is to maximize forward velocity. Ant uses state inputs. The state space is R^{37} and the action space is R^8 , including joint torques (Xing et al., 2025).

A.2. Hand Reorient

Hand Reorient is a task from Rewarped (Xing et al., 2025), where the goal is to reorient a cube with an Allegro dexterous hand, using state inputs. The state space is R^{72} and the action space is R^{16} , including joint torques (Xing et al., 2025).

A.3. Rolling Pin

Rolling Pin is a manipulation task from Rewarped (Xing et al., 2025), where the goal is to flatten dough with a rolling pin, using pixel inputs. The observation space consists of $[R^{250 \times 3}, R^3, R^3]$, and the action space is R^3 , including relative position and orientation (Xing et al., 2025).

A.4. Soft Jumper

Soft Jumper is a soft-body locomotion task from Rewarped (Xing et al., 2025) with pixel inputs. The observation space consists of $[R^{204 \times 3}, R^3, R^3, R^{222}]$, and the action space is R^{222} , including tetrahedral activations (Xing et al., 2025).

A.5. Hand Flip

Hand Flip is a soft-object manipulation task from Rewarped (Xing et al., 2025) using a Shadow Hand and pixel inputs. The observation space consists of $[R^{250 \times 3}, R^3, R^{24}]$, and the action space is R^{24} , including relative position and orientation (Xing et al., 2025).

A.6. Transport

Transport is a soft-object manipulation task from Rewarped (Xing et al., 2025) with pixel inputs. The observation space consists of $[R^{250 \times 3}, R^3, R^3, R^3]$, and the action space is R^3 , including relative position (Xing et al., 2025).

A.7. ANYmal

ANYmal is a locomotion task from DFlex (Georgiev et al., 2024) with state inputs, where the goal is to maximize the forward velocity of a quadruped robot (Hutter et al., 2017). The state space is R^{49} and the action space is R^{12} .

B. Baselines, Implementation Details and Hyperparameters

B.1. Baselines and Hyperparameters

We benchmark RFO against four baselines: SHAC (Xu et al., 2021), SAPO (Xing et al., 2025), DrAC (Wang et al., 2025b), and FlowRL (Lv et al., 2025). For SAPO, we utilize the official repository¹, from which we also adopt the improved SHAC implementation as it demonstrates improved and stable performance. For DrAC and FlowRL, we reimplement their methods based on the official codebases, extending them to support parallelized environments and visual encoders for pixel-based inputs. We retain their default samplers and integration steps for action sampling.

We tune the hyperparameters of DrAC and FlowRL individually for each task, finding that the Update-to-Data (UTD) ratio

¹<https://github.com/etaoxing/mineral>

affects their performance significantly (FlowRL defaults to 0.5, as in their official repository). Note that tuning DrAC with UTD ratios ≥ 1 is computationally prohibitive, as training would require weeks to complete. Furthermore, we observe that a higher UTD ratio does not necessarily translate to better performance. We align the neural network sizes across all algorithms. For visual input tasks, we use DP3 Point Net with layers [64, 128, 256] and 64-dimension output, average pooling for all algorithms. Shared hyperparameters are detailed in Table 3.

Table 3. Common hyperparameters for all algorithms.

	<i>shared</i>	RFO	SHAC	SAPO	DrAC	FlowRL
Horizon H	32					
Epochs for critics L		16	16	16	see UTD ratio	see UTD ratio
Epochs for actors M		1	1	1	see UTD ratio	see UTD ratio
Discount γ	0.99					
TD/GAE λ	0.95				N.A.	N.A.
Actor MLP	(400, 200, 100)					
Critic MLP	(400, 200, 100)					
Actor η		$2e-3$	$2e-3$	$2e-3$	$3e-4$	$3e-4$
Critic η	$5e-4$				$3e-4$	$3e-4$
Learning rate schedule	-	linear	linear	linear	N.A.	N.A.
Optim type	AdamW					
Optim (β_1, β_2)	(0.9, 0.999)					
Norm type	LayerNorm					
Activation type	SiLU					

 Table 4. The number of parallel environments used for each environment. These values are kept the same as in the official implementations: we follow the AHAC repository (<https://github.com/imgeorgiev/DiffRL>) for the DFlex tasks and the Rewarped repository.

	Ant	Transport	Hand Flip	Rolling Pin	Soft Jumper	Anymal	Hand Reorient
Num Envs	64	32	32	32	32	128	64

Table 5. UTD ratios for FlowRL and DrAC.

	Ant	Transport	Hand Flip	Rolling Pin	Soft Jumper	Anymal	Hand Reorient
DrAC	0.5	0.125	0.5	0.25	0.125	0.25	0.125
FlowRL	0.125	0.125	0.25	0.5	0.5	0.25	0.25

Number of Seeds and Runs. The SHAC and SAPO results on Hand Reorient and ANYmal are taken from our prior work. On Hand Reorient, all methods (RFO, DrAC, FlowRL, SHAC, SAPO) use 12 random seeds with 2 runs per seed (24 runs total per algorithm) to account for simulator stochasticity, matching the configuration of our prior work. On ANYmal, due to computational constraints, RFO, DrAC, and FlowRL use 10 seeds with a single run, while SHAC and SAPO retain the 12-seed $\times$ 2-run configuration. Despite this asymmetry, the observed performance differences are sufficiently large to support our conclusions. For the remaining tasks, all algorithms are evaluated with 10 random seeds.

n -step TD Augmentation for DrAC and FlowRL on ANYmal. We observe that the original DrAC and FlowRL implementations struggle on the ANYmal task. To enable a stronger baseline comparison, we additionally augment them with an n -step TD target on the ANYmal task, which is not present in their official repositories. We tune n and adopt $n = 10$.

B.2. Implementation Details of RFO

Since the flow velocity field can generate unbounded actions, we apply a tanh squashing function to the output. We utilize the pre-tanh values as targets for both past data CFM regularization and uniform CFM regularization (although we found that using tanh-transformed actions as targets is also effective). Additionally, we observe that excessively large pre-tanh targets can cause issues for the CFM loss; hence, we apply clamping to bound them. We clamp pre-tanh targets to a maximum absolute value of 1 for all tasks, except for HandFlip and RollingPin where we clamp to 5.

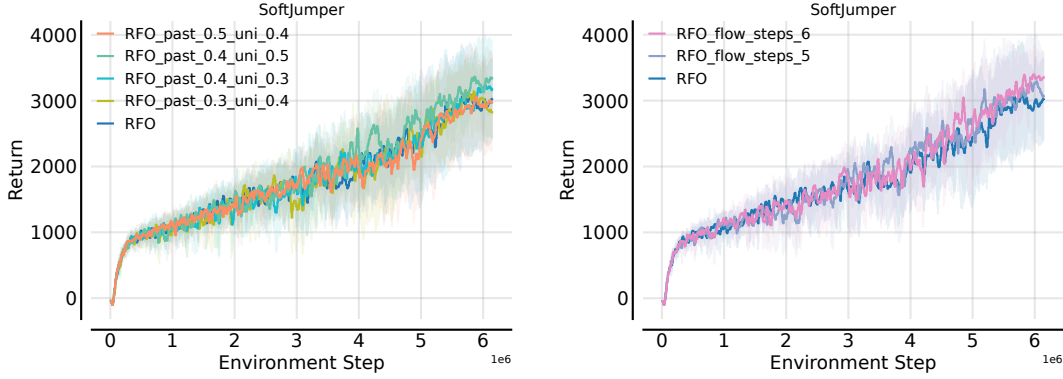


Figure 5. (a) Ablation study on different weight combinations for past data CFM regularization and uniform exploration CFM regularization. (b) Ablation study on the number of flow integration steps.

Detailed unique hyperparameters for RFO are shown in Table 6.

Table 6. RFO’s unique hyperparameters.

	Ant	Transport	Hand Flip	Rolling Pin	Soft Jumper	Anymal	Hand Reorient
c_{past}	0.2	0.1	5e-4	5e-3	0.4	0.1	0.2
c_{uni}	0.2	0.1	0.2	5e-3	0.4	0.1	0.01

C. Sensitivity to Hyperparameters

In this section, we conduct ablation experiments on: (i) different weight combinations for past data CFM regularization and uniform exploration CFM regularization, and (ii) the number of flow integration steps (RFO uses 4 flow steps across all tasks by default). As shown in Figure 5, RFO demonstrates robust performance across a wide range of hyperparameters.

D. RFO with Action Chunking Results

Figure 6 shows training curves for RFO with action chunking. Unlike standard RFO, which generates a single action per decision, this variant predicts an action chunk and uses parallel generation within the chunk (i.e., the actions in a chunk are produced jointly). Despite the harder optimization problem of predicting multi-step actions from the current observation alone, RFO with action chunking remains competitive with standard RFO on most evaluated environments. Note that its hyperparameters differ from those of standard RFO.

E. Details of KL Divergence Estimation

In this section, we detail the approximation of the KL divergence between consecutive policy updates on the Soft Jumper task. Following the methodology in FPO (McAllister et al., 2025), for a given state-action pair (s, a) and flow policies π_θ and $\pi_{\theta'}$, we can approximate the likelihood ratio $\frac{\pi_\theta(a|s)}{\pi_{\theta'}(a|s)}$ using the difference in their Conditional Flow Matching (CFM) losses. Specifically, this ratio is approximated as $\exp(L_{\theta'}^{\text{CFM}}(a|s) - L_\theta^{\text{CFM}}(a|s))$. Leveraging this relationship, we sample actions from the old/behavior policy π_θ and compute the CFM losses for both π_θ and $\pi_{\theta'}$ using these samples as targets. Finally, we employ the K3 estimator (Schulman, 2020) to estimate the KL divergence $\text{KL}(\pi_\theta \parallel \pi_{\theta'})$.

F. Design Choice

In this section, we justify our decision to include rollout states and actions from both the current and previous iterations in $\mathcal{D}_{\text{recent}}$. As shown in Figure 7, relying solely on the current iteration results in significantly slower learning. Conversely, using data from three iterations leads to degraded performance and reduced learning speed. In contrast, we find that including state-action pairs from both the current and previous iterations yields the best results in terms of final performance and

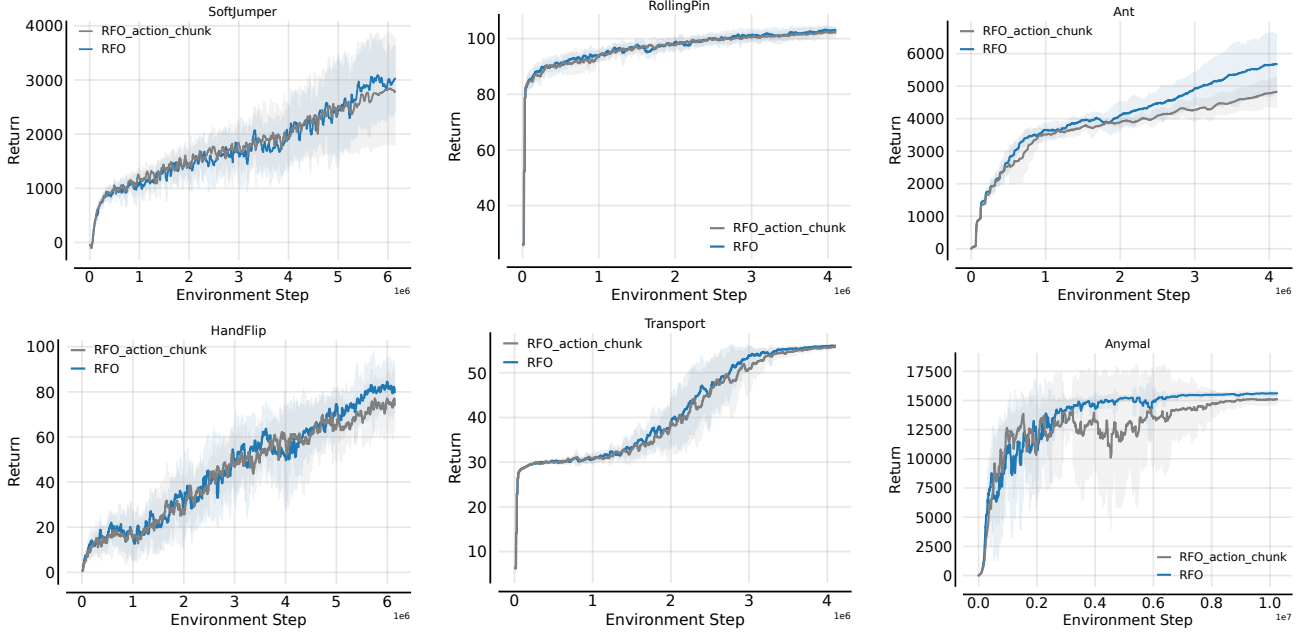


Figure 6. Training curves of RFO with the action chunking mechanism.

convergence speed.

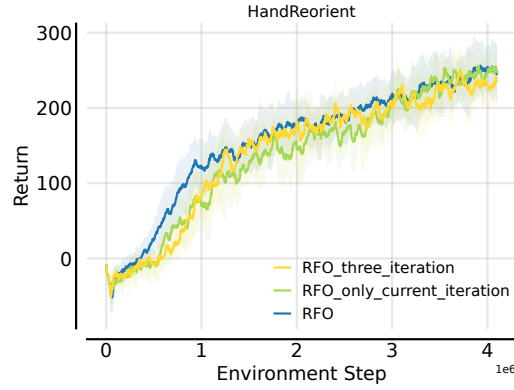


Figure 7. Comparison of $\mathcal{D}_{\text{recent}}$ including data from: the current iteration only, the current + previous iterations, and three iterations of rollout data.